CHAPTER 8

Testing in a Dockerized Environment

In the previous chapter, we completed our standard app setup by adding a Postgres database. However, we're not quite done with development yet. As professional Ruby developers, we value well-tested code that gives us confidence that we're delivering reliable software. As we rebuild our development environment around Docker, we need to find out how testing fits into the picture. Whatever your personal preferences around testing, it's important to know how to get our testing tools working and playing nice with Docker so that you can use them as needed.

In this chapter, we're going to set up the popular Ruby testing framework: RSpec. I've chosen RSpec over Rails' default—Minitest—for a couple of reasons. First, slightly more effort is needed to set up RSpec, so there's more to learn along the way. Second, it happens to be my testing framework of choice for Rails projects.

That said, if you're an ardent Minitest fan, never fear. It's still worth reading this chapter to continue to build up your familiarity with the Docker commands needed in our normal workflow. Also, the configuration required, especially for Capybara, will be very similar and may largely be translated across to Minitest.

If you've ever set up RSpec on a Rails project before, the majority of what you'll see in this chapter will be very familiar. In fact, squint and you could miss the fact that we're using Docker at all. This is both a testament to how far you've come in your learning and understanding of Docker, and to the fact that, once set up, the Docker tools get out of your way and fade into the background until you need them.

Despite that, this chapter is not without its challenges. There are a few nuances to testing with Docker that we'll tease out. We'll also see that things